

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



NAMA : Ruhfaidah Shirotul Akhiroh

NIM : 103132400019

KELAS : S1DS-REG-05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Abstraksi

Menentukan hal-hal yang esensial pada suatu objek dan mengabaikan hal-hal yang sifatnya insidental. Seperti menentukan ciri-ciri (atribut) yang dimiliki oleh objek tersebut dan apa saja yang dapat dilakukan (metode) oleh objek.

2. Enkapsulasi

Proses pemaketan data objek bersama method-methodnya. Manfaat utama pengkapsulan yaitu menyembunyian rincian implementasi dari objek lain yang tidak berhak.

Fungsi dari enkapsulasi:

a. Penyembunyian data

Objek disusun dari antarmuka *public Method* dan *private* data. Manfaat utama adalah bagian internal dapat berubah tanpa mempengaruhi bagian-bagian program lain.

b. Modularitas

Kode sumber bagian internal objek dikelola secara terpisah dari antarmuka, maka bebas melakukan modifikasi yang tidak menyebabkan masalah pada bagian-bagian lain.

3. Inheritance

Proses penciptaan kelas baru dengan mewarisi karakteristik dari kelas yang telah ada, ditambah karakteristik unik kelas baru itu. Jika superclass telah mendefinisikan perilaku yang diperlukan, maka tidak perlu mendefinisi ulang perilaku itu, cukup membuat kelas yang merupakan *subclass* dari *superclass* yang dimaksud.

4. Polymorphisms

Konsep ini memungkinkan digunakan antarmuka yang sama untuk memerintah suatu objek agar dapat melakukan aksi yang mungkin secara prinsip sama tapi secara proses berbeda. Mekanisme *polymorphism* dapat dilakukan dengan beberapa cara, seperti *Overloading Method*, *Overloading Constructor*, maupun *Overring Method*.

5. Kelas

Kelas dapat mendeklarasikan variabel dan menciptakan *Object*. Sebuah kelas mempunyai anggota yang terdiri atas atribut dan method. Dikatakan fungsi bila *method* tersebut melakukan suatu proses dan mengembalikan suatu nilai (return value), dan dikatakan prosedur bila *method* tersebut hanya melakukan suatu proses dan tidak mengembalikan nilai (void).

```
class <classname>
{
```

```
//declaration of data member
//declaration of Methods
}
```

6. Object

Insatansiasi atau hasil ciptaan dari suatu kelas yang telah dibuat sebelumnya. Dalam pengembangan program orientasi objek lebih lanjut, sebuah objek dimungkinkan terdiri atas objek-objek lain.

7. Field

Dapat berupa variabel kelas, variabel objek, variabel method objek atau parameter dari sebuah fungsi. Field merupakan anggota dari kelas yang digunakan untuk menyimpan data. Terdapat dua field dalam kelas;

a. Class field

Field yang dikaitkan dengan kelas dimana dia didefinisikan, bukan dengan sebuah *instance* dari kelas.

b. Instance field

Field apapun yang dideklarasikan tanpa menggunakan *static modifier*.

8. Method

Dalam OOP, Method digunakan untuk memodularisasi program melalui pemisahan tugas dalam suatu kelas. Pemanggilan Method menspesifikasikan nama Method dan menyediakan informasi (parameter) yang diperlukan untuk melaksanakan tugasnya. Deklarasi Method untuk yang mengembalikan nilai (fungsi);

```
[modifier] Type-data namaMethod(parameter1,parameter2,...parameterN)
{
    Deklarasi-deklarasi dan proses ;
    return nilai-kembalian
}
```

Deklarasi Method untuk yang tidak mengembalikan nilai (prosedur).

```
[[modifier] void namaMethod(parameter1,parameter2,...parameterN)
{
    Deklarasi-deklarasi dan proses ;
}
```

Terdapat dua cara melewati argumen ke Method, yaitu:

a. Melewatkan secara nilai (Pass by Value)

Digunakan untuk argumen yang mempunyai tipe data primitif (Byte, short, int, long, float, double, char, dan boolean).

b. Melewatkan secara referensi (Pass by Reference)

Digunakan pada array dan objek. Prosesnya isi memory pada variabel array dan objek merupakan penunjuk ke alamat memory yang mengandung data sesungguhnya yang akan dioperasikan.

9. Keyword “this”

Digunakan pada array dan objek. Prosesnya isi memory pada variabel array dan objek merupakan penunjuk ke alamat memory yang mengandung data sesungguhnya yang akan dioperasikan.

B. Guided

1. Abstraksi

```
package Guided03.Abstraksi;

public class Kunci {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + " " +ras + " " +"meow");
    }
}
```

```
package Guided03.Abstraksi;

public class Abstraksi {
    public static void main(String[] args){
        Kunci Pissi = new Kunci();

        Pissi.nama = "Pissi sisi";
        Pissi.ras = "Oyen";
        Pissi.bersuara();
    }
}
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ projec
- Pissi sisi Oyen meow
-----
BUILD SUCCESS
-----
Total time:  1.790 s
Finished at: 2026-10-01T19:41:18+07:00
-----
```

2. Enkapsulasi

```
package Guided03.Enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo anda: " + saldo);
    }
}
```

```
package Guided03.Enkapsulasi;

public class Enkapsulasi {
    public static void main(String[] arg){
        Rekening rek = new Rekening();

        rek.tambahSaldo(150000);
        rek.tampilkanSaldo();
        rek.tambahSaldo(300000);
        rek.tampilkanSaldo();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ proj
saldo berhasil ditambahkan
Saldo anda: 150000
saldo berhasil ditambahkan
Saldo anda: 450000
-----
BUILD SUCCESS
-----
Total time: 1.050 s
Finished at: 2026-10-01T19:42:45+07:00
```

3. Field

```
package Guided03.Field;

public class Circle {
    public static final double PI = 3.14159;
```

```

    public static double radiansToDegrees(double rads){
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference(){
        return 2 * PI * r;
    }
}

```

Output :

Tidak ada output

4. Pass by Value & Pass by Reference

```

package Guided03.Method;

public class TestPass {
    int i, j;

    TestPass(int a, int b) {
        i = a;
        j = b;
    }
    //passed by value dengan parameter berupa tipe data primitif

    void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
    }
    //passed by reference dengan berupa tipe data class

    void calculate(TestPass e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
    }
}

```

```

package Guided03.Method;

public class PassedByValue {
    public static void main(String[] args) {

```

```

int x, y;
TestPass z;
z = new TestPass(50, 100);
x = 10;
y = 20;
System.out.println("Nilai sebelum passed by value : ");
System.out.println("x = " + x);
System.out.println("y = " + y);
//passed by value
z.calculate(x, y);
System.out.println("Nilai sesudah passed by value : ");
System.out.println("x = " + x);
System.out.println("y = " + y);
System.out.println("Nilai sebelum passed by reference : ");
System.out.println("z.i = " + z.i);
System.out.println("z.j = " + z.j);
//passed by reference
z.calculate(z);
System.out.println("Nilai sesudah passed by reference : ");
System.out.println("z.i = " + z.i);
System.out.println("z.j = " + z.j);
}
}

```

Output :

```

--- exec:3.6.3:exec (default-cli) @ projectModul3 --
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
-----
BUILD SUCCESS
-----
Total time:  1.656 s
Finished at: 2026-10-01T19:52:12+07:00
-----

```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

```
package com.week3.projectmodul3.UnGuided;

public class PengolahSuhu {
    private double[] suhuHarian;
    private static final double NilaiKosong = -1.0;
    public PengolahSuhu(double[] suhuHarian){
        this.suhuHarian = suhuHarian; //menyimpan referensi array yang sama
    }

    public void tampilkanData(){
        for (int i = 0; i < suhuHarian.length; i++){
            if (suhuHarian[i] == NilaiKosong) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + " C");
            }
        }
    }

    public int cariIndexKosong(){
        for (int i = 0; i < suhuHarian.length; i++){
            if (suhuHarian[i] == NilaiKosong){
                return i;
            }
        }
        return -1;
    }

    public void isiDataKosong(){
```

```

        int i = cariIndexKosong();
        if (i != -1){
            suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) / 2;
        }
    }

    public double hitungRataRata(){
        double total = 0;
        for (double suhu : suhuHarian) {
            total += suhu;
        }
        return total / suhuHarian.length;
    }
}

```

```

package com.week3.projectmodul3.UnGuided;

import java.util.Arrays;
import java.util.Locale;

public class GoSuhu {
    public static void main(String[] args){
        double[] suhuHarian = {30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " +
            pengolah.cariIndexKosong());
        System.out.println();

        pengolah.isiDataKosong();

        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();

        System.out.println(String.format(Locale.US, "Rata-rata : %.2f C",
            pengolah.hitungRataRata()));
        System.out.println();

        System.out.println("Isi array suhu Harian di main setelah isiDataKosong()
            dijalankan: ");
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi array
            yang sama)");
    }
}

```

```
}
```

Output :

```
=== Data Suhu Awal ===
Hari 1 : 30.4 C
Hari 2 : 24.3 C
Hari 3 : 26.8 C
Hari 4 : (kosong)
Hari 5 : 31.4 C
Hari 6 : 30.8 C
Hari 7 : 32.9 C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4 C
Hari 2 : 24.3 C
Hari 3 : 26.8 C
Hari 4 : 29.1 C
Hari 5 : 31.4 C
Hari 6 : 30.8 C
Hari 7 : 32.9 C

Rata-rata : 29.39 C

Isi array suhu Harian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
-----
BUILD SUCCESS
-----
Total time: 1.903 s
Finished at: 2026-10-03T22:51:02+07:00
```

Penjelasan :

Terdapat 2 *class* yaitu class *PengolahSuhu* dan *GoSuhu* (sebagai main).

1. *PengolahSuhu*

- a. Atribut *suhuHarian* bertipe array *double[]* dan bersifat *private* sehingga hanya dapat diakses dari dalam *class* ini sendiri (enkapsulasi). *NilaiKosong* adalah konstanta bernilai -1.0 digunakan sebagai penanda “data suhu harian belum terisi”, memakai nilai static sehingga nilainya sama untuk semua objek.
- b. Constructor
Constructor menerima array *suhuHarian* dari luar dan menyimpan ke atribut *this.suhuHarian*.
- c. Method *tampilkanData()*

Melakukan perulangan *for* dari index 0 sampai akhir array. Untuk setiap elemen, akan dicek apakah nilainya sama dengan *NilaiKosong* (-1.0), jika iya maka ditampilkan sebagai “kosong”, jika tidak ditampilkan nilai suhunya dengan satuan C. $(i + 1)$ digunakan agar penomoran hari dimulai dari 1 bukan 0.

d. Method *cariIndexKosong()*

Mencari index pertama dalam array yang nilainya *NilaiKosong*. Begitu ditemukan akan return *i* (menghentikan loop). Jika loop selesai tanpa menemukan data kosong, method mengembalikan nilai -1.

e. Method *isiDataKosong()*

Memanggil *cariIndexKosong()* untuk tahu posisi data yang kosong, jika ada ($i \neq -1$) nilai kosong itu diisi dengan rata-rata dari suhu harian sebelum dan sesudahnya.

f. Method *hitungRataRata()*

Menggunakan enhanced for-loop untuk menjumlahkan semua elemen array ke variabel *total*. Hasil akhirnya dibagi dengan jumlah elemen (*suhuHarian.length*) untuk mendapatkan rata-rata.

2. GoSuhu

- Membuat array suhu 7 hari, dengan hari ke 4 (index 3) bernilai -1.0 (data kosong).
- Membuat objek *pengolah* dari class *PengolahSuhu*, dan mengoper array *suhuHarian* ke constructor.
- Menampilkan data suhu awal (hari ke 4 menampilkan “kosong”) dan index data yang kosong.
- Memanggil *isiDataKosong()* sehingga *suhuHarian* pada index yg kosong diisi otomatis dengan rata-rata *suhuHarian[2]* dan *suhuHarian[4]*. Kemudian menampilkan ulang data yang hari ke 4 sudah terisi.
- Menghitung dan mencetak rata-rata suhu setelah data lengkap. *Locale.US* digunakan supaya format desimal selalu pakai titik sebagai pemisah, bukan koma.
- Terakhir akan menampilkan *suhuHarian* dengan format menggunakan *Arrays.toString()*.

D. Kesimpulan

Walaupun perubahan data dilakukan melalui objek, array yang dideklarasikan di main ikut berubah. Ini membuktikan bahwa array di Java bersifat *pass by reference* dan *pass by value* sehingga objek tidak menyimpan salinan array, melainkan menunjuk ke array fisik yang sama persis di memory dengan yang ada di main.